

Лабораторная работа №3. Маршрутизация, параметры пути и query-параметры

1. Краткое содержание

Изучение механизмов маршрутизации в веб-фреймворках. Освоение параметров пути (`req.params`), `query`-параметров (`req.query`), вложенных маршрутов, `wildcard`-маршрутов и валидации входных параметров. Создание REST-подобного API с гибкой маршрутизацией на основе данных из ЛР №2.

2. Цель работы: Освоить различные способы передачи данных через URL. Научиться строить гибкие маршруты с параметрами пути и `query`-параметрами, комбинировать их, обрабатывать вложенные ресурсы и валидировать входные данные. Понять разницу между `req.params`, `req.query` и `req.body`.

3. Теоретическое обоснование

3.1. Что такое маршрутизация

Маршрутизация (`routing`) - это механизм сопоставления HTTP-запроса (метод + URL) с кодом-обработчиком на сервере. Каждый маршрут определяет, какой код выполнится при обращении клиента по конкретному адресу.

В Express маршрут описывается как `app.METHOD(PATH, HANDLER)`,

где `METHOD` - HTTP-метод (`get`, `post`, `put`, `delete`, `patch`), `PATH` - путь URL (может содержать параметры), `HANDLER` - функция-обработчик, принимающая `req` и `res`.

Во Flask маршрут задаётся декоратором

@app.route('/path', methods=['GET']) над функцией-обработчиком.

3.2. Три источника данных в запросе

При обработке запроса сервер может получать данные из трёх мест:

Источник	Где находится	Express	Flask
Параметры пути	Часть URL: /users/42	req.params.id	<int:user_id>
Query-параметры	После ?: /users?age=20	req.query.age	request.args.get('age')
Тело запроса	Тело POST/PUT/PATCH	req.body	request.json

Важно: параметры пути и query-параметры передаются в URL, тело запроса - отдельно (для POST/PUT/PATCH).

3.3. Параметры пути (route parameters)

Параметр пути - это именованная часть URL, которая извлекается сервером и передаётся в обработчик. Пример: GET /users/42, где параметр id = 42.

Express:

```
app.get('/users/:id', (req, res) => {  
  const id = req.params.id; // "42"  
  res.json({ userId: id });  
});
```

Flask:

```
@app.route('/users/<int:user_id>')  
def get_user(user_id):  
  return jsonify({"userId": user_id})
```

Особенности:

1. параметр всегда строка в Express (нужен `parseInt`);
2. во Flask можно указать тип: `<int:user_id>`, `<string:name>`, `<float:price>`;
3. параметров может быть несколько:
`/posts/:postId/comments/:commentId`;
4. параметры могут быть необязательными: `/users/:id?` (Express 4.x+) или `@app.route('/users/', defaults={'id': None})` (Flask).

3.4. Query-параметры (query string)

Query-параметры - это пары «ключ=значение», передаваемые после ? в URL. Они используются для фильтрации, сортировки, пагинации, поиска.

Пример: GET

`/cities?search=Москва&sort=population&order=desc&page=1&limit=10`

Express:

```
app.get('/cities', (req, res) => {  
  const { search, sort, order, page, limit } = req.query;  
  // search = "Москва", sort = "population", order = "desc", page = "1",  
  limit = "10"  
});
```

Flask:

```
@app.route('/cities')  
def get_cities():  
  search = request.args.get('search')  
  sort = request.args.get('sort', default='id')  
  page = request.args.get('page', default=1, type=int)
```

Особенности:

1. query-параметры всегда строки (нужно приводить типы);
2. могут быть множественные значения: `?tag=js&tag=node`;

3. отсутствие параметра - undefined в Express, None во Flask;
4. значения по умолчанию нужно задавать явно.

3.5. Вложенные маршруты (nested routes)

Вложенные маршруты отражают иерархию ресурсов: один ресурс принадлежит другому.

GET /users/42/posts	- все посты пользователя 42
GET /users/42/posts/7	- пост 7 пользователя 42
GET /posts/7/comments	- комментарии поста 7
GET /posts/7/comments/3	- комментарий 3 поста 7

Express:

```
app.get('/users/:userId/posts', (req, res) => {
  const { userId } = req.params;
  // ...
});

app.get('/users/:userId/posts/:postId', (req, res) => {
  const { userId, postId } = req.params;
  // ...
});
```

Flask:

```
@app.route('/users/<int:user_id>/posts')
def user_posts(user_id):
    ...

@app.route('/users/<int:user_id>/posts/<int:post_id>')
def user_post(user_id, post_id):
    ...
```

3.6. Wildcard-маршруты и приоритеты

Wildcard - маршрут, который соответствует любому пути. Используется для обработки 404 или для SPA.

Express:

```
// Порядок важен! Wildcard - последним
app.get('/users/:id', (req, res) => { /* ... */ });
app.get('/users/:id/posts', (req, res) => { /* ... */ });
app.use((req, res) => {
  res.status(404).json({ error: 'Маршрут не найден' });
});
```

Flask:

```
@app.errorhandler(404)
def not_found(error):
    return jsonify({"error": "Маршрут не найден"}), 404
```

Правило: более специфичные маршруты должны идти раньше более общих. Express проверяет маршруты в порядке объявления.

3.7. Валидация параметров

Параметры приходят как строки, поэтому нужна валидация:

```
app.get('/cities/:id', (req, res) => {
  const id = parseInt(req.params.id);

  if (isNaN(id) || id < 1) {
    return res.status(400).json({
      error: 'ID должен быть положительным числом'
    });
  }

  const city = cities.find(c => c.id === id);
  if (!city) {
    return res.status(404).json({ error: 'Город не найден' });
  }

  res.json(city);
});
```

Что валидировать:

id - целое положительное число;
 page, limit - целые положительные, $\text{limit} \leq 100$;
 sort - из белого списка полей;
 order - asc или desc;
 search - строка, не слишком длинная.

3.8. Сравнение: params vs query vs body

Критерий	req.params	req.query	req.body
Где в запросе	В пути URL	После ?	В теле
Когда использовать	Идентификация ресурса	Фильтрация, поиск	Создание/обновление
Методы	GET, PUT, DELETE	GET	POST, PUT, PATCH
Пример	/users/42	/users?age=20	{"name": "Иван"}
Тип	Всегда строка	Всегда строка	Любой (JSON)
Обязательность	Часто обязателен	Обычно опционален	Зависит от метода

4. Пример выполнения

Вариант 1. Node.js + Express

Шаг 1. Подготовка проекта

```
mkdir lab3-routing-params
cd lab3-routing-params
npm init -y
npm install express
npm install --save-dev nodemon
```

Шаг 2. Настройка package.json

```
"scripts": {
```

```
"start": "node app.js",  
"dev": "nodemon app.js"  
}
```

Шаг 3. Создание сервера (app.js)

```
const express = require('express');  
const app = express();  
const port = 3000;  
  
app.use(express.json());  
  
app.use((req, res, next) => {  
  console.log(`[${new Date().toISOString()}  
    ] ${req.method  
  } ${req.url  
  }`);  
  next();  
});  
  
// ===== ДАННЫЕ =====  
let cities = [  
  { id: 1, name: 'Москва', population: 13000000, country: 'Россия', area:  
2561  
  },  
  { id: 2, name: 'Санкт-Петербург', population: 5600000, country: 'Россия',  
area: 1439  
  },  
  { id: 3, name: 'Новосибирск', population: 1600000, country: 'Россия',  
area: 505  
  },  
  { id: 4, name: 'Екатеринбург', population: 1500000, country: 'Россия',  
area: 1112  
  },  
  { id: 5, name: 'Казань', population: 1250000, country: 'Россия', area: 425  
  }  
];  
  
// ===== БАЗОВЫЙ УРОВЕНЬ =====  
// GET /cities - все города  
app.get('/cities', (req, res) => {  
  res.json({ count: cities.length, cities  
  });  
});  
  
// GET /cities/:id - один город  
app.get('/cities/:id', (req, res) => {
```

```

    const id = parseInt(req.params.id);

    if (isNaN(id)) {
        return res.status(400).json({ error: 'ID должен быть числом'
    });
    }

    const city = cities.find(c => c.id === id);
    if (!city) {
        return res.status(404).json({ error: 'Город не найден'
    });
    }

    res.json(city);
});

// GET /cities/:id/area - площадь города
app.get('/cities/:id/area', (req, res) => {
    const id = parseInt(req.params.id);
    const city = cities.find(c => c.id === id);

    if (!city) {
        return res.status(404).json({ error: 'Город не найден'
    });
    }

    res.json({
        city: city.name,
        area: city.area,
        unit: 'км²'
    });
});

// GET /countries/:country/cities - города страны
app.get('/countries/:country/cities', (req, res) => {
    const country = req.params.country;
    const filtered = cities.filter(c =>
        c.country.toLowerCase() === country.toLowerCase()
    );

    res.json({
        country,
        count: filtered.length,
        cities: filtered
    });
});

// ===== СРЕДНИЙ УРОВЕНЬ =====

```

```

// GET /cities?search=&sort=&order=&page=&limit=
app.get('/cities', (req, res) => {
  let result = [...cities];

  // Поиск по названию
  const search = req.query.search;
  if (search) {
    result = result.filter(c =>
      c.name.toLowerCase().includes(search.toLowerCase())
    );
  }
  // Фильтр по стране
  const country = req.query.country;
  if (country) {
    result = result.filter(c =>
      c.country.toLowerCase() === country.toLowerCase()
    );
  }
  // Сортировка
  const sort = req.query.sort;
  const order = req.query.order || 'asc';
  const allowedSortFields = ['id', 'name', 'population', 'area'];

  if (sort) {
    if (!allowedSortFields.includes(sort)) {
      return res.status(400).json({
        error: `Поле сортировки должно быть одним из:
${allowedSortFields.join(', ')}`
      });
    }
    if (!['asc', 'desc'].includes(order)) {
      return res.status(400).json({
        error: 'Параметр order должен быть asc или desc'
      });
    }
  }

  result.sort((a, b) => {
    if (order === 'desc') {
      return a[sort] < b[sort] ? 1 : -1;
    }
    return a[sort] > b[sort] ? 1 : -1;
  });
});

```

```

    ] ? 1 : -1;
  });
}
// Пагинация
const page = parseInt(req.query.page) || 1;
const limit = parseInt(req.query.limit) || 10;

if (page < 1) {
  return res.status(400).json({ error: 'page должен быть >= 1' });
}
if (limit < 1 || limit > 100) {
  return res.status(400).json({ error: 'limit должен быть от 1 до 100' });
}

const start = (page - 1) * limit;
const end = start + limit;
const paginated = result.slice(start, end);

res.json({
  count: result.length,
  page,
  limit,
  totalPages: Math.ceil(result.length / limit),
  cities: paginated
});
});

// ===== ПРОДВИНУТЫЙ УРОВЕНЬ =====
// GET /cities/:id/details - детальная информация с вложенными данными
app.get('/cities/:id/details', (req, res) => {
  const id = parseInt(req.params.id);
  const city = cities.find(c => c.id === id);

  if (!city) {
    return res.status(404).json({ error: 'Город не найден' });
  }
  // Имитация вложенных данных
  const districts = [
    { id: 1, name: 'Центральный', cityId: id },
    { id: 2, name: 'Северный', cityId: id }
  ];

  res.json({

```

```

        city,
        districts,
        attractions: ['Кремль', 'Парк', 'Музей'
        ]
    });
});

// GET /countries/:country/cities/:cityId - город в стране
app.get('/countries/:country/cities/:cityId', (req, res) => {
    const { country, cityId
    } = req.params;
    const id = parseInt(cityId);

    const city = cities.find(c =>
        c.id === id &&
        c.country.toLowerCase() === country.toLowerCase()
    );

    if (!city) {
        return res.status(404).json({
            error: `Город с ID=${cityId
            } не найден в стране ${country
            }`
        });
    }

    res.json({
        country,
        city
    });
});

// ===== ОБРАБОТКА 404 =====
app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден'
    });
});

app.listen(port, () => {
    console.log(`Сервер запущен на http: //localhost:${port}`);
});

```

Примечание. В приведённом фрагменте `app.get('/cities', ...)` размещён дважды (базовый и расширенный обработчики). При реальном запуске вызовы следует объединить в один обработчик либо использовать `next()`

согласованно, чтобы избежать дублирования маршрута. Ниже в разделе 4.1 приведён единый рабочий вариант.

Шаг 4. Запуск сервера

npm run dev

4.1. Итоговый объединённый обработчик /cities

```
// GET /cities - базовый список + поиск + сортировка + пагинация
app.get('/cities', (req, res) => {
  let result = [...cities];

  const search = req.query.search;
  if (search) {
    result = result.filter(c =>
      c.name.toLowerCase().includes(search.toLowerCase())
    );
  }

  const country = req.query.country;
  if (country) {
    result = result.filter(c =>
      c.country.toLowerCase() === country.toLowerCase()
    );
  }

  const sort = req.query.sort;
  const order = req.query.order || 'asc';
  const allowedSortFields = ['id', 'name', 'population', 'area'];

  if (sort) {
    if (!allowedSortFields.includes(sort)) {
      return res.status(400).json({
        error: `Поле сортировки должно быть одним из:
${allowedSortFields.join(', ')}`
      });
    }
  }
  if (!['asc', 'desc'].includes(order)) {
    return res.status(400).json({ error: 'Параметр order должен быть asc или desc' });
  }
});
```

```

    }
    result.sort((a, b) => (order === 'desc'
      ? (a[sort] < b[sort] ? 1 : -1)
      : (a[sort] > b[sort] ? 1 : -1)));
  }

  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;

  if (page < 1) return res.status(400).json({ error: 'page должен быть >= 1' });
  if (limit < 1 || limit > 100) {
    return res.status(400).json({ error: 'limit должен быть от 1 до 100' });
  }

  const start = (page - 1) * limit;
  const paginated = result.slice(start, start + limit);

  res.json({
    count: result.length,
    page,
    limit,
    totalPages: Math.ceil(result.length / limit),
    cities: paginated
  });
});

```

4.2. Проверка (Postman / браузер)

№	Запрос	Ожидаемый результат
1	GET http://localhost:3000/cities	Все города
2	GET http://localhost:3000/cities/1	Москва
3	GET http://localhost:3000/cities/999	404 Not Found
4	GET http://localhost:3000/cities/abc	400 Bad Request

5	GET http://localhost:3000/cities/1/area	Площадь Москвы
6	GET http://localhost:3000/countries/Россия/cities	Города России
7	GET http://localhost:3000/cities?search=Москва	Поиск
8	GET http://localhost:3000/cities?sort=population&order=desc	Сортировка
9	GET http://localhost:3000/cities?page=1&limit=2	Пагинация
10	GET http://localhost:3000/cities/1/details	Детали с вложенными
11	GET http://localhost:3000/countries/Россия/cities/1	Город в стране
12	GET http://localhost:3000/anything-else	404

Вариант 2. Python + Flask

Шаг 1. Подготовка проекта

```
mkdir lab3-routing-params
```

```
cd lab3-routing-params
```

```
python -m venv venv
```

```
source venv/bin/activate    # Linux/Mac
```

```
# venv\Scripts\activate    # Windows
```

```
pip install flask
```

Шаг 2. Создание сервера (app.py)

```
from flask import Flask, jsonify, request
import time

app = Flask(__name__)
```

```

cities = [
    {
        "id": 1,
        "name": "Москва",
        "population": 13000000,
        "country": "Россия",
        "area": 2561
    },
    {
        "id": 2,
        "name": "Санкт-Петербург",
        "population": 5600000,
        "country": "Россия",
        "area": 1439
    },
    {
        "id": 3,
        "name": "Новосибирск",
        "population": 1600000,
        "country": "Россия",
        "area": 505
    },
    {
        "id": 4,
        "name": "Екатеринбург",
        "population": 1500000,
        "country": "Россия",
        "area": 1112
    },
    {
        "id": 5,
        "name": "Казань",
        "population": 1250000,
        "country": "Россия",
        "area": 425
    }
]

@app.before_request
def log_request():
    print(f"[{time.strftime('%Y-%m-%d %H:%M:%S')}] {request.method} {request.path}")

# ===== БАЗОВЫЙ УРОВЕНЬ =====

@app.route('/cities')
def get_cities():

```

```

    return jsonify({
        "count": len(cities),
        "cities": cities
    })

@app.route('/cities/<int:city_id>')
def get_city(city_id):
    city = next((c for c in cities if c[
        "id"
    ] == city_id), None)
    if not city:
        return jsonify({
            "error": "Город не найден"
        }),
404
    return jsonify(city)

@app.route('/cities/<int:city_id>/area')
def get_city_area(city_id):
    city = next((c for c in cities if c[
        "id"
    ] == city_id), None)
    if not city:
        return jsonify({
            "error": "Город не найден"
        }),
404
    return jsonify({
        "city": city[
            "name"
        ],
        "area": city[
            "area"
        ],
        "unit": "км²"
    })

@app.route('/countries/<string:country>/cities')
def get_country_cities(country):
    filtered = [c for c in cities if c[
        "country"
    ].lower() == country.lower()]
    return jsonify({
        "country": country,
        "count": len(filtered),
        "cities": filtered
    })

```

```

# ===== СРЕДНИЙ УРОВЕНЬ =====

@app.route('/cities/search')
def search_cities():
    result = list(cities)

    search = request.args.get('search')
    if search:
        result = [c for c in result if search.lower() in c[
            "name"
        ].lower()]

    country = request.args.get('country')
    if country:
        result = [c for c in result if c[
            "country"
        ].lower() == country.lower()]

    sort = request.args.get('sort')
    order = request.args.get('order', 'asc')

    if sort:
        allowed = ['id', 'name', 'population', 'area']

        if sort not in allowed:
            return jsonify({
                "error": f"sort должен быть одним из: {allowed}"
            }),
400

        if order not in ['asc', 'desc']
]:
            return jsonify({
                "error": "order должен быть asc или desc"
            }),
400

        result.sort(key=lambda x: x[sort
], reverse=(order == 'desc'))

    page = request.args.get('page',
1, type=int)
    limit = request.args.get('limit',
10, type=int)

    if page < 1:
        return jsonify({

```

```

        "error": "page должен быть >= 1"
    }),
    400
    if limit < 1 or limit > 100:
        return jsonify({
            "error": "limit должен быть от 1 до 100"
        }),
        400

    start = (page - 1) * limit
    end = start + limit
    paginated = result[start:end
]

    return jsonify({
        "count": len(result),
        "page": page,
        "limit": limit,
        "totalPages": (len(result) + limit - 1) // limit,
        "cities": paginated
    })

# ===== ПРОДВИНУТЫЙ УРОВЕНЬ =====

@app.route('/cities/<int:city_id>/details')
def get_city_details(city_id):
    city = next((c for c in cities if c[
        "id"
] == city_id), None)
    if not city:
        return jsonify({
            "error": "Город не найден"
        }),
        404

    districts = [
        {
            "id": 1,
            "name": "Центральный",
            "cityId": city_id
        },
        {
            "id": 2,
            "name": "Северный",
            "cityId": city_id
        }
    ]

    return jsonify({

```

```

        "city": city,
        "districts": districts,
        "attractions": [
            "Кремль",
            "Парк",
            "Музей"
        ]
    })

@app.route('/countries/<string:country>/cities/<int:city_id>')
def get_country_city(country, city_id):
    city = next((c for c in cities if c[
        "id"
    ] == city_id and c[
        "country"
    ].lower() == country.lower()), None)
    if not city:
        return jsonify({
            "error": f"Город с ID={city_id} не найден в стране {country}"
        }),
        404
    return jsonify({
        "country": country,
        "city": city
    })

# ===== ОБРАБОТКА 404 =====

@app.errorhandler(404)
def not_found(error):
    return jsonify({
        "error": "Маршрут не найден"
    }),
    404

if __name__ == '__main__':
    app.run(port=3000, debug=True)

```

Шаг 3. Запуск

python app.py

5. Индивидуальные задания

Задание 1. Базовый уровень (max 30 баллов)

Требования:

1. 4 эндпоинта с параметрами пути: GET /items (все элементы), GET /items/:id (один элемент), GET /items/:id/<related> (вложенный ресурс), GET /<category>/items (элементы категории);
2. обработка 400 при нечисловом id;
3. обработка 404 при отсутствии элемента;
4. настроен автоматический перезапуск (nodemon / debug=True).

Вариант	Сущность	Поля	Вложенный ресурс	Категория
1	Задачи (tasks)	id, title, done	/tasks/:id/comments	/projects/:projectId/tasks
2	Пользователи (users)	id, name, email	/users/:id/posts	/cities/:city/users
3	Товары (products)	id, name, price	/products/:id/reviews	/categories/:cat/products
4	Книги (books)	id, title, author	/books/:id/chapters	/authors/:author/books
5	Фильмы (movies)	id, title, year	/movies/:id/actors	/genres/:genre/movies
6	Студенты (students)	id, name, group	/students/:id/grades	/groups/:group/students
7	Города (cities)	id, name, population	/cities/:id/districts	/countries/:country/cities
8	Сотрудники (employees)	id, name, position	/employees/:id/tasks	/departments/:dept/employees
9	Курсы (courses)	id, title, duration	/courses/:id/lessons	/levels/:level/courses
10	Автомобили (cars)	id, brand, model	/cars/:id/owners	/brands/:brand/cars
11	Рецепты (recipes)	id, title, time	/recipes/:id/steps	/difficulties/:diff/recipes
12	Игры (games)	id, title, genre	/games/:id/reviews	/platforms/:platform/games
13	Отели (hotels)	id, name, stars	/hotels/:id/rooms	/cities/:city/hotels
14	Рестораны (restaurants)	id, name, cuisine	/restaurants/:id/dishes	/cuisines/:cuisine/restaurants
15	Песни (songs)	id, title, artist	/songs/:id/lyrics	/artists/:artist/songs

Задание 2. Средний уровень (max 45 баллов)

Требования: всё из базового + query-параметры:

GET /items?search=... - поиск по названию;

GET /items?sort=field&order=asc|desc - сортировка;

GET /items?page=1&limit=10 - пагинация;

GET /items?filter=field:value - фильтрация по полю.

Валидация:

sort - только из белого списка полей;

order - только asc/desc;

page ≥ 1 , limit от 1 до 100;

при невалидных - 400 Bad Request.

Ответ пагинации включает: count, page, limit, totalPages, items.

Вариант	Сущность	Поля сортировки	Поля фильтрации
1	Задачи	id, title, priority, deadline	done, priority
2	Пользователи	id, name, age	city, age
3	Товары	id, name, price, stock	category, price
4	Книги	id, title, year	genre, year
5	Фильмы	id, title, year, rating	director, rating
6	Студенты	id, name, avg_grade	group, course
7	Города	id, name, population, area	country, population
8	Сотрудники	id, name, salary	department, salary
9	Курсы	id, title, duration, price	level, price
10	Автомобили	id, brand, year, price	brand, year
11	Рецепты	id, title, time	difficulty, time
12	Игры	id, title, rating	platform, genre
13	Отели	id, name, stars, price_per_night	city, stars
14	Рестораны	id, name, rating	cuisine, rating
15	Песни	id, title, duration	artist, album

Задание 3. Продвинутый уровень (max 60 баллов)

Требования: всё из среднего + дополнительные функции:

1. вложенные маршруты: GET /categories/:catId/items, GET /categories/:catId/items/:itemId;
2. множественные параметры пути: GET /users/:userId/posts/:postId/comments/:commentId;
3. wildcard-маршрут для обработки 404;
4. валидация всех параметров с понятными сообщениями об ошибках;
5. консольное логирование всех запросов (метод + URL + время + IP);
6. дополнительный эндпоинт GET /items/stats - статистика (количество, среднее, min/max);
1. обработка 500 - глобальный обработчик ошибок.

Вариант	Сущность 1	Сущность 2	Статистика	Связь
1	Задачи	Проекты	Кол-во выполненных	По приоритету
2	Пользователи	Компании	Средний возраст	По городу
3	Товары	Бренды	Средняя цена	По категории
4	Книги	Авторы	Кол-во по жанрам	По автору
5	Фильмы	Кинотеатры	Средний рейтинг	По режиссёру
6	Заказы	Клиенты	Сумма заказов	По статусу
7	Студенты	Курсы	Средний балл	По группе
8	Посты	Комментарии	Кол-во комментариев	По автору
9	Тикеты	События	Кол-во открытых	По приоритету
10	Сотрудники	Офисы	Средняя зарплата	По отделу
11	Рецепты	Шефы	Среднее время	По сложности
12	Матчи	Команды	Кол-во побед	По команде
13	Номера	Отели	Средняя цена	По типу
14	Блюда	Меню	Средняя калорийность	По категории
15	Автомобили	Дилеры	Средняя цена	По марке

6. Контрольные вопросы

Базовый уровень

1. Что такое маршрутизация? Как она работает в Express/Flask?
2. Чем отличаются `req.params`, `req.query` и `req.body`?
3. Как получить параметр пути в Express? Во Flask?
4. Что произойдёт, если не преобразовать параметр пути в число?
5. Как обработать ситуацию, когда параметр пути отсутствует?
6. Что такое вложенные маршруты? Приведите пример.
7. Какой код ответа возвращается, если ресурс не найден?
8. Какой код ответа возвращается при невалидном параметре?
9. Как объявить необязательный параметр пути в Express?
10. Как работает порядок объявления маршрутов в Express?
Почему он важен?

Средний уровень

11. Как получить `query`-параметры в Express/Flask? Чем они отличаются от параметров пути?
12. Как задать значение по умолчанию для `query`-параметра?
13. Как реализовать поиск по подстроке через `query`-параметр?
14. Как реализовать сортировку по нескольким полям?
15. Как реализовать пагинацию с подсчётом общего количества страниц?
16. Как валидировать `query`-параметры? Какие коды ответов использовать?
17. Как реализовать фильтрацию по нескольким полям одновременно?
18. Что произойдёт, если `query`-параметр передан несколько раз (`?tag=js&tag=node`)?
19. Как ограничить максимальное значение `limit`?
1. Как использовать белый список полей для сортировки?

Продвинутый уровень

- 20.Как комбинировать параметры пути и query-параметры в одном запросе?
- 21.Как реализовать вложенные маршруты с несколькими уровнями?
- 22.Как обработать wildcard-маршрут и почему он должен быть последним?
- 23.Как реализовать глобальный обработчик ошибок?
- 24.Как логировать все запросы с IP-адресом клиента?
- 25.Как защитить API от слишком больших значений limit?
- 26.Как реализовать статистику по коллекции?
- 27.Как организовать структуру проекта при росте числа маршрутов?
- 28.Как документировать параметры маршрутов (OpenAPI/Swagger)?
- 29.Как тестировать маршруты с параметрами в Postman?

7. Требования к отчёту

Отчёт должен быть представлен в формате Markdown и содержать:

Титульный лист - название работы, ФИО, группа, вариант, технология.

Цель работы - формулировка цели.

Теоретическое обоснование - краткое изложение (2–3 абзаца): маршрутизация, req.params, req.query, вложенные маршруты, валидация.

Выполнение практического примера - код сервера, скриншоты всех запросов через Postman/Thunder Client/браузер, команды для запуска.

Выполнение индивидуального задания - код сервера, скриншоты всех эндпоинтов, включая обработку ошибок (400, 404).

Ответы на контрольные вопросы - не менее 5 по своему уровню.

Вывод - что было сделано, что нового узнали, с какими трудностями столкнулись.

Список использованных источников - ссылки на документацию.

8. Критерии оценивания

Критерий	Базовый (max 30)	Средний (max 45)	Продвинутый (max 60)
Настройка среды и авто-перезапуск	5	5	5
Базовые маршруты с параметрами пути	15	10	5
Вложенные маршруты	5	10	10
Валидация параметров (400)	5	5	5
Обработка 404	-	5	5
Query-параметры (поиск, сортировка, пагинация)	-	10	5
Множественные параметры пути	-	-	5
Wildcard-маршрут	-	-	5
Консольное логирование	-	-	5
Дополнительный эндпоинт (stats)	-	-	5
Глобальный обработчик ошибок (500)	-	-	5
Итого	30	45	60

Шкала перевода баллов в оценку

Оценка	Базовый (30)	Средний (45)	Продвинутый (60)
5 (отлично)	-	-	54–60
4 (хорошо)	-	40–45	46–53
3 (удовлетворительно)	24–30	34–39	38–45
2 (неудовлетворительно)	0–23	0–33	0–37

9. Структура репозитория

student-backend-course/

- |— lab1-first-http-server/
- |— lab2-http-methods/
- |— lab3-routing-params/
 - | |— README.md
 - | |— app.js / app.py
 - | |— package.json / requirements.txt
 - | |— Lab3_Routing.postman_collection.json
 - | |— screenshots/
 - | |— cities-all.png
 - | |— city-by-id.png
 - | |— city-404.png
 - | |— city-area.png
 - | |— country-cities.png
 - | |— search.png
 - | |— sort.png
 - | |— pagination.png
 - | |— nested.png
 - | |— error-500.png
- |— lab4-sql-basics/
- |— ...

10. Рекомендуемые источники

Express - Routing - <https://expressjs.com/en/guide/routing.html>

Express - Request - <https://expressjs.com/en/4x/api.html#req>

Flask - Routing -

<https://flask.palletsprojects.com/en/latest/quickstart/#routing>

Flask - Request - <https://flask.palletsprojects.com/en/latest/api/#incoming-request-data>

HTTP-методы (MDN) -
<https://developer.mozilla.org/ru/docs/Web/HTTP/Methods>

URL-параметры (MDN) -
[https://developer.mozilla.org/ru/docs/Learn/Common_questions/What is a URL](https://developer.mozilla.org/ru/docs/Learn/Common_questions/What_is_a_URL)

REST API Tutorial - <https://restfulapi.net/>

Postman Learning Center - <https://learning.postman.com/>